Programación con Objetos 3

Presentación de la materia

Autores:

- Facundo Javier Gelatti — javiergelatti@gmail.com

Este documento está bajo una licencia CC BY 4.0

Índice

Sobre esta materia **1**
- ¿De qué trata esta materia? 1
  - Una introducción preliminar 1
  - Un resumen 2
  - Contenidos 2
- Algunos temas operativos 4
  - Instancias y criterios de evaluación 4
  - Dinámica de las clases 5
- La materia en el contexto de la carrera 5
  - El perfil de las personas egresadas 6
  - ¿Cómo se relaciona esto con la materia? 7

Última modificación: *2026-03-21*

Sobre esta materia

¡Hola! Si estás leyendo esto, es porque estás cursando Programación con Objetos III. Estamos en una etapa avanzada de la carrera, tanto de la Tecnicatura en Programación (en caso de que estés cursando Objetos 3 como complementaria) como de la Licenciatura en Informática de la UNQ. Haber llegado hasta aquí implica que has cursado muchas otras materias, en particular las dos anteriores sobre programación con objetos (y varias otras más si la cursás como materia obligatoria de la licenciatura).

Durante la cursada vamos a aprovechar todo ese recorrido.

¿De qué trata esta materia?

Una introducción preliminar

Para empezar, es fundamental ubicarnos en el contexto de la carrera. Cada materia tiene su *programa*, que incluye:

1. sus objetivos y contenidos mínimos (definidos por el plan de estudios de la carrera); y
2. un listado de unidades, bibliografía y descripción de la organización de las clases (definidos por el equipo docente inicial y refinados cada cierto tiempo).

Es importante que siempre que vayan a cursar una materia, tengan presente el contenido de su programa[1], en especial el punto 1.

Dicho esto, hay que saber que la información que se encuentra en el programa no siempre refleja de manera exacta lo que sucede en la cursada. Esto puede pasar por distintas razones, por ejemplo, porque parte de la información esté desactualizada (en especial con respecto al punto 2), o por la coyuntura de cada cuatrimestre y cómo afecta eso a la cantidad de días de clases.

Sin embargo, lo importante es que tengas presente cómo se concibió la materia en el contexto del plan de estudios, cómo se vincula con otras materias de la carrera y qué es lo que se supone que tenés que aprender. Saber esto te va a permitir compararlo con lo que luego efectivamente ocurra en la cursada, identificar las decisiones que tomamos los docentes sobre la cursada, formar una postura al respecto, consultar sobre estos temas durante las clases y dar retroalimentación.

Con eso en mente, hablemos sobre el enfoque actual de la materia y veamos qué tenemos pensado para esta cursada.

[1] El programa de Programación con Objetos III puede consultarse desde http://cpi.blog.unq.edu.ar/materias/obj3/

Un resumen

Al pensar en el nombre de la materia, podemos notar que nos indica 3 cosas[2]:

1. que vamos a hablar sobre *programación*,
2. que lo haremos en el contexto del *paradigma orientado a objetos*, y
3. que, sobre ambas cosas, *ya tenemos un recorrido previo*.

Si bien vamos a estar trabajando en el marco de la programación orientada a objetos, pensamos que es importante que durante la materia tratemos algunas ideas que van más allá de este paradigma; de ahí la distinción entre “programación” y “objetos” (1 y 2).

Al revisar el programa, vemos que el objetivo principal de la materia es que quienes cursen sean capaces de utilizar tecnologías y conceptos que *extienden el modelo orientado a objetos clásico*, incluyendo aquellos de uso frecuente en la industria del desarrollo de software.

En las materias anteriores de la carrera se trabaja con un modelo de objetos "tradicional" (basado en clases, con herencia simple, etc.). En Objetos 3 vamos a ir más allá de eso, explorando variantes, extensiones y herramientas que amplían lo que se puede hacer con objetos. Además, vamos a ver algunos conceptos que son de uso habitual en los lenguajes y entornos de trabajo actuales, en particular en lo que respecta a la construcción de herramientas de programación.

Sobre esto último, vale aclarar que no nos vamos a limitar a los conceptos que sean populares en la industria, sino que vamos a pensar también en *qué alternativas son posibles*, aún si hoy nadie las usa. Los lenguajes y las herramientas que usamos no son inevitables: fueron diseñados tomando ciertas decisiones, y esas decisiones podrían haber sido otras. Parte de pensar críticamente sobre la programación es no dar por supuesto que lo que conocemos es lo único que puede existir, y en esta materia queremos ejercitar esa capacidad de imaginar alternativas y cuestionar lo establecido.

❦

Para explorar estas cuestiones, vamos a necesitar herramientas concretas. Ahora pasamos a ver, entonces, los contenidos de la materia y cómo van a estar organizados.

Contenidos

La materia está organizada en dos grandes partes, cada una con un lenguaje de programación y un trabajo práctico asociado.

En la primera parte de la cursada vamos a trabajar con *Ruby*, un lenguaje orientado a objetos dinámicamente tipado. Los temas principales van a ser:

[2] El hecho (ocurrente) de que el nombre “Programación con Objetos III” nos haga pensar en **3** cosas es un detalle potencialmente autorreferencial. Esto lo hace, a su vez, *doblemente* autorreferencial, porque la autorreferencia es un concepto del que vamos a estar hablando durante la cursada.

- **Esquemas no tradicionales para definir el comportamiento de un objeto.** En particular, vamos a estudiar *mixines* y *traits*, que son mecanismos complementarios a la herencia simple. También vamos a ver cómo modelar usando *closures*, es decir, bloques de código representados como objetos.

- **Metaprogramación.** La metaprogramación consiste en escribir programas *cuyo dominio es la programación*: programas que manipulan, inspeccionan o generan otros programas (o, posiblemente, a sí mismos). Vamos a trabajar con el metamodelo de Ruby, explorando capacidades como la introspección, y la modificación de clases y métodos en tiempo de ejecución.

En la segunda parte de la cursada vamos a trabajar con *Scala*, un lenguaje estáticamente tipado que combina un enfoque orientado a objetos con el paradigma funcional. Nos vamos a centrar en:

- **Conceptos avanzados de tipado.** Vamos a estudiar qué son los sistemas de tipos para un lenguaje orientado a objetos, qué posibilidades y consecuencias tiene el uso del tipado estático, y conceptos como subtipado, tipado nominal vs. estructural, e inferencia de tipos.

- **Programación mixta objetos-funcional.** Vamos a explorar cómo se combinan herramientas del paradigma de objetos con herramientas del paradigma funcional para resolver problemas concretos, analizando en cada caso las ventajas y desventajas de cada enfoque.

Algo importante para remarcar es que “aprender Ruby y Scala” no es el objetivo de la materia. Los lenguajes van a ser simplemente las herramientas que vamos a usar para explorar los conceptos. Hemos elegido estos lenguajes porque cada uno de ellos tiene características que nos permiten entrar en los temas que nos interesan: Ruby es especialmente expresivo para hacer metaprogramación, mientras que Scala ofrece un sistema de tipos avanzado y está pensado para combinar el paradigma orientado a objetos y el paradigma funcional.

Al margen: una paradoja de la programación

El hecho de tener que utilizar lenguajes concretos y querer profundizar en ellos, pero siendo a la vez fundamental que los conceptos que veamos vayan más allá de estos lenguajes particulares, puede enunciarse como una *paradoja*, como lo hicieron Martínez López et al. (2012):

> *[...] una forma en la que podemos enunciar esta paradoja es:*
>
> “El lenguaje de programación que utilizamos no es importante,
> pero es extremadamente importante”
>
> *Lo que queremos indicar es que el lenguaje que utilizamos no es importante, porque debemos enfocarnos en las ideas a estudiar antes que en las*

> *características únicas de un lenguaje; pero a la vez es importante, porque es el único medio para poder describir, conceptualizar y comunicar ideas, por lo que debemos aprender a dominarlo adecuadamente. Cada tipo de lenguaje posee características propias, que encauzan la forma en que describimos ideas. Sumando esto a que en un mismo lenguaje pueden existir diversas formas de materializar una idea, es necesario concentrarse en la esencia de dicha idea para que el lenguaje no se vuelva un obstáculo que nos aleje del entendimiento de la misma.*

Vamos a tomar esto en cuenta durante la cursada, y a explorar algunos ejemplos en otros lenguajes, para poder entender mejor cómo pensar más allá de las herramientas que estaremos usando.

Algunos temas operativos

Instancias y criterios de evaluación

Comentamos brevemente qué tendrán que realizar, y cómo los estaremos evaluando.

Trabajos prácticos

Durante la cursada se realizan **dos trabajos prácticos grupales** (en grupos de 3 o 4 personas). Son trabajos con una complejidad significativa, pensados para desarrollarse a lo largo de varias semanas.

Cada TP cuenta con:

- *Checkpoints*: instancias de seguimiento previas a la entrega, en las que cada grupo se reúne con el equipo docente para mostrar su avance, resolver dudas y recibir retroalimentación. Es importante tener avances y dudas concretas para aprovechar cada instancia.
- *Entrega y defensa*: una instancia donde se presenta el trabajo terminado y se defienden las decisiones de diseño tomadas.
- *Recuperatorio*: una instancia adicional para corregir lo que haya quedado pendiente tras la entrega.

Los trabajos prácticos requieren que tengan una dedicación sostenida. Es fundamental no dejar pasar las primeras semanas sin familiarizarse con el lenguaje correspondiente (esto es, instalar el entorno y resolver los primeros ejercicios), ya que ese tiempo perdido después se paga caro al momento de encarar el TP.

En la entrega y defensa vamos a evaluar tanto la funcionalidad implementada como la calidad interna de la solución (es decir, la solución debe funcionar *pero también* estar bien diseñada e implementada).

También evaluaremos el avance en los checkpoints, valorando un ritmo sostenido en el progreso y el aprovechamiento de las instancias de seguimiento.

Ejercicios individuales

Además de los TPs, se realizan **ejercicios individuales** más pequeños a lo largo de la cursada. Estos ejercicios están pensados para entrar en calor con los conceptos y el lenguaje antes de encarar cada TP, y deberían poder resolverse en no más de una semana. Si bien el núcleo de la evaluación son los TPs, los ejercicios individuales son una parte importante de la preparación y también forman parte de la evaluación.

Evaluaciones teóricas

Cada parte de la materia tendrá su **evaluación teórica** correspondiente, que se realizará de manera individual.

Valoración del desempeño general

Esto incluye la participación en clase, la actitud y la honestidad. Esperamos que participen activamente, hagan preguntas y aprovechen el espacio de la clase para que podamos construir el conocimiento de manera colectiva. Para poder participar, es fundamental que lleven al día el trabajo con el material.

Dinámica de las clases

Las clases son teórico-prácticas. Combinan la exposición de conceptos con ejercicios y actividades en clase. Se espera y se fomenta la participación activa: hacer preguntas, plantear dudas, proponer alternativas, cuestionar lo que estamos diciendo. El valor de la clase está en la interacción y en el espíritu crítico.

Cualquier sugerencia o comentario para mejorar la materia es bienvenido en cualquier momento de la cursada.

❦

La materia en el contexto de la carrera

Para finalizar, pensemos en cómo Objetos 3 se integra con el contexto de la carrera. Tomaremos como referencia el plan de estudios de la Licenciatura en Informática, actualizado por última vez en la resolución 128/19 del Consejo Superior de la UNQ[3]. Así como mencionamos la importancia de conocer el programa de cada materia, es igualmente importante conocer cómo está estructurada la carrera.

[3] Pueden encontrarlo en:
http://cpi.blog.unq.edu.ar/wp-content/uploads/sites/14/2020/03/plan_de_estudios_li_2019.pdf

El perfil de las personas egresadas

Revisando el plan de estudios con detenimiento, veremos que Objetos 3 forma parte de una carrera cuyo eje central es *la problemática de la construcción de software*. De la sección *Perfil del Egresado*:

> *[...] profesional universitario cuya área de acción principal es la problemática de la construcción de software, incluyendo todas las fases involucradas en el proceso.*

Pensemos un segundo en esto: ¿qué implica *la problemática de la construcción de software, incluyendo todas las fases del proceso*? ¿Tiene que ver únicamente con aspectos técnicos? ¿Puede pensarse de manera más amplia? ¿En tu experiencia, existen ya buenas soluciones a todos los problemas en la construcción de software? ¿Cómo podríamos pensar en alternativas? ¿Qué tipo de cosas podrían *inventarse*?

Por otro lado, veamos algunas otras características de los profesionales que la carrera busca formar:

> *[...] será capaz de diseñar soluciones a problemas informáticos como así también implementar dichas soluciones describiendo los conceptos que fundamentan las decisiones que tomó [...].*

Es decir, no solamente hay que diseñar soluciones efectivas, sino también poder describir sus fundamentos y justificar nuestras decisiones.

Además, también se hace mención al trabajo en equipo:

> *Podrá conformar y liderar equipos de trabajo que aborden estas problemáticas y que privilegien la colaboración por sobre la competencia [...]".*

Vale la pena detenerse en esta idea. La carrera propone que seamos capaces de trabajar en equipos *que privilegien la colaboración por sobre la competencia*. Nosotros pensamos que esto es algo muy importante. Por un lado, porque hay resultados y pensamientos a los que solo puede llegarse cuando se dan estas condiciones, y que de otra manera serían inaccesibles. Por otro lado, poder trabajar y pensar en conjunto de manera efectiva es algo que es muy sano para el entorno en el que estamos, y hasta es más divertido y estimulante cuando se logra trabajar así.

Poder llegar a eso es un desafío, tanto para nosotros (docentes) como para ustedes (estudiantes). Desde Objetos 3, vamos a intentar incentivar la generación de ideas, los debates y los desarrollos en conjunto. Sin embargo, no podemos estar presentes en las micro-dinámicas internas de cada grupo, por lo que preocuparse por estos temas también será responsabilidad de ustedes (por ejemplo, es una idea poco provechosa intentar dividir el TP en partes independientes al inicio, y repartirlas de manera uniforme entre los miembros del grupo).

Por último, algunos de los objetivos de la carrera que son especialmente relevantes para esta materia son (el énfasis es nuestro):

- Que el egresado tenga *pensamiento crítico*.
- Que sea *creativo e innovador*.
- Que adquiera *conocimientos robustos* sobre los procesos de análisis, diseño, programación e implantación de software.
- Que adquiera *experiencia práctica* en la aplicación de esos conocimientos.

Creemos que todas estas cosas se relacionan entre sí: tener conocimientos robustos y experiencia práctica es lo que habilita a la creatividad y al pensamiento crítico. Nos recuerda a la frase atribuida a Pablo Picasso: “Aprendé las reglas como un profesional, para que puedas romperlas como un artista”.

¿Cómo se relaciona esto con la materia?

Es nuestra intención que todos estos objetivos de la carrera se reflejen directamente en la forma en la que planteamos Objetos 3. Se trata de desarrollar la capacidad de *analizar críticamente* las herramientas y decisiones de diseño, *experimentar* con distintas alternativas, y *fundamentar* las decisiones que tomemos.

La materia propone un espacio de discusión y debate donde se evalúan ventajas y desventajas de los distintos enfoques, se comparan lenguajes y paradigmas, y se construye el conocimiento de manera colaborativa.

¡Esperamos que disfrutes de la cursada!

❦

Referencias

Martínez López, P. E., Bonelli, E. A., & Sawady O'Connor, F. A. (2012). El nombre verdadero de la programación. Una concepción de enseñanza de la programación para la sociedad de la información. En *Simposio sobre la Sociedad de la Información (SSI), JAIIO* (Vol. XLI). SADIO. http://sedici.unlp.edu.ar/handle/10915/124718